

LCP-Nav: Latent Consequence Prediction for Cross-Domain Visual Navigation

Zhonghua An*, Jianxiang Liu*, Yunzhao Yang*, Ningyan Guo*[†]

**Beijing University of Posts and Telecommunications*

Beijing, China

2294822424@qq.com; liujianxiang@bupt.edu.cn

2025140284@bupt.cn; guoningyan@bupt.edu.cn

[†]Corresponding author: Ningyan Guo

Abstract—Visual navigation policies can make steady route progress yet accumulate enough local error to miss an image-specified destination. Policies that directly regress one local trajectory provide no explicit comparison among plausible actions that terminate at different locations. We present Latent Consequence Prediction for Navigation (LCP-Nav), which proposes a finite set of trajectories, rolls each proposal through shared action-conditioned latent dynamics, and selects the predicted endpoint most consistent with the goal. Cross-domain tests show lower action loss than ViNT on ReCon and SCAND, while repeated Gazebo experiments show lower endpoint error and more validated arrivals across three routes. LCP-Nav uses 26.77 M parameters and reaches a p95 latency of 24.8 ms on an RTX 4090, sustaining inference above 40 Hz. The experiments show that latent endpoint comparison is an effective and efficient decision layer for visual navigation.

Index Terms—visual navigation, world models, latent dynamics, mobile robots

I. INTRODUCTION

Visual navigation guides a mobile robot to an image-specified destination from recent RGB observations and a goal image. Modern policies learn this mapping from heterogeneous robot trajectories and transfer it across platforms and environments [2], [3]. Their success is commonly summarized by route progress or local action error. Neither quantity, however, directly describes where the robot finishes. Small deviations that appear harmless over one prediction horizon can accumulate, leaving a robot far from its destination after it has traversed most of the route.

This mismatch is especially relevant to policies that directly regress a single local trajectory. Such a policy can learn a strong average action, but its output does not expose alternative motions or distinguish candidates by their predicted terminal states. Our closed-loop audit reveals the practical consequence: high topological progress can coexist with substantial endpoint error. Improving arrival therefore requires a decision variable tied to the consequence of an action sequence, rather than another estimate of local progress.

Two local trajectories can be similarly plausible under imitation loss while leading to states with different relationships to the goal. Once one is executed, its displacement and heading error enter the next planning cycle. Retaining several feasible actions until their likely endpoints have been compared gives the policy a chance to avoid that error before execution.

World models offer a natural route to consequence-aware decisions, but generating complete future RGB observations is unnecessary when the decision only requires a comparison among local actions [8], [9]. A compact latent transition can instead predict how an action sequence changes the task-relevant visual state. For this prediction to guide selection, two conditions must hold: different actions must produce distinguishable latent endpoints, and the selector must use those endpoints in relation to the goal.

LCP-Nav is designed around these conditions. A deterministic proposal head exposes K feasible local trajectories; shared action-conditioned dynamics map each proposal to an imagined endpoint; and a goal-aware scorer selects among the resulting consequences. Soft candidate supervision keeps proposal and ranking targets aligned, while SIGReg preserves variation in the latent state so that endpoint comparisons remain informative. Future observations supervise the dynamics during training, but inference uses the same observation history and goal image as a standard visual policy.

The candidate head creates alternatives without iterative sampling, and the shared transition evaluates them in one representation space. The goal–endpoint difference represents terminal compatibility directly; non-collapse regularization keeps visually distinct states separable. LCP-Nav therefore changes how local actions are selected while retaining the original visual inputs and topological navigation interface.

We test each link in this design. Matched offline experiments measure in-domain and cross-domain action prediction; recursive-rollout and endpoint-permutation tests determine whether the learned dynamics remain predictive and influence selection; repeated Gazebo runs measure endpoint error and validated arrival; and profiling quantifies the cost of finite latent rollout. The resulting evidence connects the proposed decision mechanism to both transfer and closed-loop behavior.

II. RELATED WORK

General visual-navigation policies. ViNG introduced visual navigation with topological memory [1]; GNM standardized heterogeneous robot data [2]; and ViNT scaled a Transformer policy across robots [3]. NoMaD models multimodal actions with diffusion and goal masking [4], while LiMo augments limited demonstrations with planner-generated supervision [7].

These approaches improve transfer, data efficiency, or action diversity, but their action output is not selected through an explicit prediction of each candidate’s endpoint. Evaluations of zero-shot navigation and collision avoidance further motivate decisions that remain reliable under visual and geometric shifts [5], [6].

World models and latent prediction. Navigation World Models generate action-conditioned future views for planning [8], and UniWM couples visual foresight with autoregressive action generation [9]. ExploreVLA uses future RGB and depth as dense supervision for autonomous driving [10]. LCP-Nav predicts only the latent endpoint needed to rank a small deterministic candidate set, eliminating pixel reconstruction and iterative action sampling. Related joint-embedding models stabilize prediction in representation space [11]; here, latent dynamics are trained jointly with distance estimation, trajectory proposal, and endpoint-based selection.

III. METHOD

A. Problem and Architecture

We formulate local planning as selection among predicted consequences. At time t , the policy receives $O_t = \{o_{t-P}, \dots, o_t\}$ and a goal image o_g . Observation and goal encoders form tokens that a Transformer fuses into h_t . A candidate head predicts

$$\{\hat{A}_t^1, \dots, \hat{A}_t^K\} = f_{\text{cand}}(h_t), \quad (1)$$

where each candidate contains H waypoints. Each waypoint is $a_i = (\Delta x_i, \Delta y_i, \cos \theta_i, \sin \theta_i)$. EfficientNet-B0 encodes the six observations and the current-goal image pair. Type embeddings distinguish history, current-observation, and goal tokens, while sinusoidal positional encoding preserves their order before four Transformer encoder layers. Their final observation and goal tokens define z_t and z_g , while mean token h_t drives the candidate and temporal-distance heads. The proposal head combines h_t with learned hypothesis identities, predicts bounded position increments and normalized headings, and accumulates the increments into local-frame waypoints. Figure 1 separates deployment from training-only supervision.

B. Propose, Imagine, and Select

For candidate k , the trajectory error combines Smooth-L1 position error and orientation cosine distance, $e_k = e_k^{\text{pos}} + \lambda_\theta e_k^\theta$. Candidate regression and ranking use soft targets

$$w_k = \frac{e^{-e_k/\tau_a}}{\sum_j e^{-e_j/\tau_a}}, \quad q_k = \frac{e^{-e_k/\tau_s}}{\sum_j e^{-e_j/\tau_s}}, \quad (2)$$

with $L_{\text{reg}} = \sum_k w_k e_k$ and $L_{\text{score}} = -\sum_k q_k \log \text{softmax}(s)_k$. The complete candidate objective is

$$L_{\text{cand}} = L_{\text{reg}} + L_{\text{score}} + 0.1L_{\text{smooth}} + 0.05L_{\text{step}} + 0.02L_{\text{div}}. \quad (3)$$

The smoothness and step terms suppress erratic motion, while L_{div} separates candidate endpoints. Soft assignment avoids abrupt changes in the winning candidate and gives the scorer a ranking target consistent with trajectory quality.

Regression distributes supervision by candidate quality instead of updating only a hard winner, and the score loss learns the same ordering. The diversity margin acts on final positions, preserving alternative endpoints without forcing every intermediate waypoint to differ. All candidates are produced in one forward pass.

The shared transition predicts $\hat{z}_{t+1} = F_{\text{dyn}}(z_t, a_t)$ and recursively rolls out each candidate,

$$\hat{z}_{t+h}^k = F_{\text{dyn}}(\hat{z}_{t+h-1}^k, \hat{a}_{t+h-1}^k). \quad (4)$$

The scorer receives $[z_t, z_g, \hat{z}_{t+H}^k, z_g - \hat{z}_{t+H}^k, \phi_A(\hat{A}^k)]$, where ϕ_A is a trajectory encoder, and deployment selects $k^* = \arg \max_k s_k$. The goal-endpoint difference makes terminal compatibility directly available to the selector. All candidates share transition parameters, so their consequences are compared in one representation space.

F_{dyn} is a residual multilayer perceptron with LayerNorm. Recursive application evaluates each complete proposal with one transition shared across time and candidates. The scorer combines geometric information from ϕ_A with the visual relation between the imagined endpoint and goal.

C. Predictive Training and Non-Collapse Regularization

During training, expert actions drive a three-step teacher-forced rollout and real future frames provide stop-gradient targets. With validity mask $m_{b,h}$, the dynamics loss is

$$L_{\text{dyn}} = \frac{\sum_{b,h} m_{b,h} [1 - \cos(\hat{z}_{b,t+h}, \text{sg}(z_{b,t+h}))]}{\sum_{b,h} m_{b,h}}. \quad (5)$$

Prediction alone can collapse distinct states. SIGReg instead matches one-dimensional random projections of observation and future tokens to a standard Gaussian. For embeddings $Z \in \mathbb{R}^{B \times T \times D}$, unit projections r_m , knots u_l , and $W = \sum_l \omega_l$, we compute

$$L_{\text{sig}} = \frac{1}{MW} \sum_{m,l} \omega_l [(\mathbb{E} \cos(u_l Z^\top r_m) - e^{-u_l^2/2})^2 + (\mathbb{E} \sin(u_l Z^\top r_m))^2]. \quad (6)$$

The expectation averages batch and temporal tokens. Matching projected characteristic functions prevents a low prediction loss from being obtained with a near-constant representation. The complete training objective is

$$L = \bar{L}_{\text{dist}} + \bar{L}_{\text{cand}} + \gamma(e) \bar{L}_{\text{dyn}} + \lambda_{\text{sig}} L_{\text{sig}}, \quad (7)$$

where bars denote division by an exponential moving average of the corresponding loss magnitude and $\gamma(e)$ warms the dynamics term by epoch. L_{dist} is Smooth-L1 error on temporal goal distance. Training uses future observations only as latent targets; deployment requires the observation history, goal image, and candidate actions.

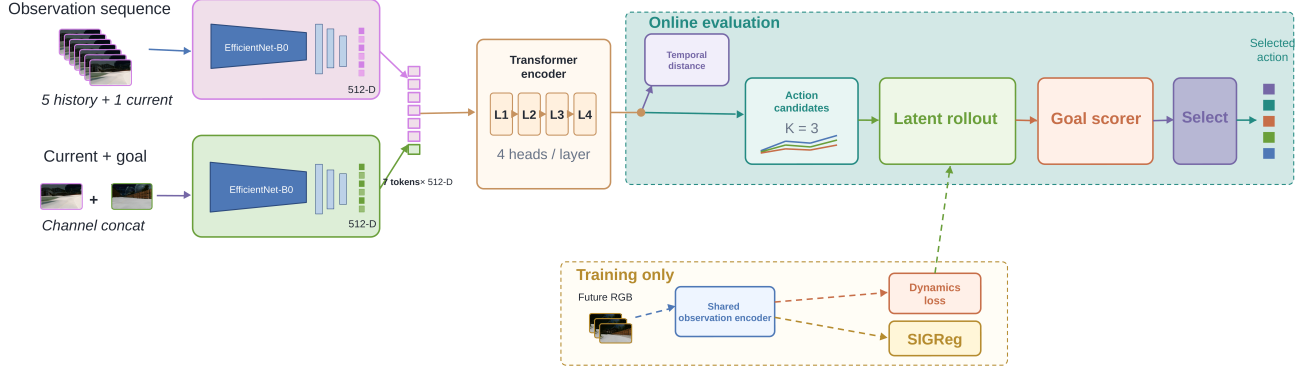


Fig. 1. LCP-Nav architecture. Six observation tokens and one current-goal fusion token enter a four-layer Transformer. Its shared representation feeds a ViNT-style temporal-distance head and an action head that produces three candidate trajectories. Latent dynamics predicts the consequence of each candidate, and the goal-conditioned scorer selects the executed action. During training, future RGB frames pass through the same observation encoder; the dynamics loss trains the shared latent rollout, while SIGReg regularizes observation and future embeddings. Solid arrows denote online inference; amber dashed arrows denote training-only supervision.

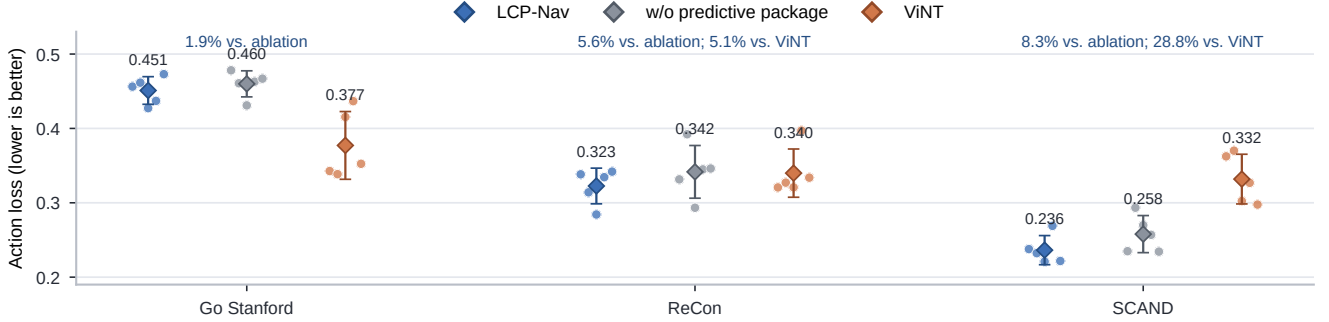


Fig. 2. Five-seed offline action loss (points) with mean $\pm$ sample standard deviation (diamonds and bars). All checkpoints were selected on Go Stanford validation data; ReCon and SCAND are checkpoint-only zero-shot tests. The predictive-pathway ablation removes imagined endpoints, latent rollout, and dynamics supervision under otherwise matched training.

IV. EXPERIMENTS

A. Experimental Setup

Offline evaluation. LCP-Nav, a matched predictive-pathway ablation, and ViNT are trained independently with five seeds on Go Stanford. The ablation retains the inputs, backbone, action interface, data, and optimization budget but removes imagined endpoints, action-conditioned rollout, and dynamics supervision. Validation-selected checkpoints are evaluated on fixed Go Stanford, ReCon, and SCAND samples; ReCon and SCAND receive no fine-tuning or target-domain reselection. Each complete training run is one replicate ($n = 5$), and seed-level results define the uncertainty in Fig. 2.

All models use 96×96 RGB inputs, six observation frames, five predicted waypoints, 512-dimensional visual features, and four Transformer encoder layers. LCP-Nav uses $K = 3$, $\lambda_\theta = 1$, $\tau_a = \tau_s = 1$, $\lambda_{\text{sig}} = 0.12$, a three-step supervised dynamics horizon, 1024 SIGReg projections, and 17 characteristic-function knots. The dynamics weight is zero for the first five epochs, increases linearly to one over epochs 5–10,

and remains one thereafter. AdamW trains for 15 epochs with batch size 128, an initial learning rate of 8×10^{-4} , and cosine scheduling. Data, test samples, and optimization budgets are matched. Offline action loss averages waypoint Smooth-L1 and orientation cosine losses.

Validation action loss selects LCP-Nav-action and ViNT for action comparisons. Validation latent-prediction loss selects LCP-Nav-LFP for rollout analysis and Gazebo deployment. Test-set and Gazebo results are not used in checkpoint selection.

Closed-loop evaluation. Gazebo experiments use fixed indoor-obstacle, forest-road, and campus-road routes to test constrained passage, appearance variation, and long-horizon drift. Each method completes five runs per route. Camera, chassis, controller, topological graph, starting poses, and arrival detector are held fixed. A validated arrival requires software-reported arrival, endpoint error no greater than 5 m, and no collision or timeout. Endpoint error is the Euclidean distance to the demonstration endpoint; topological progress is the largest reached node index divided by the target-node index. Each replicate is a complete run, including collisions and timeouts.

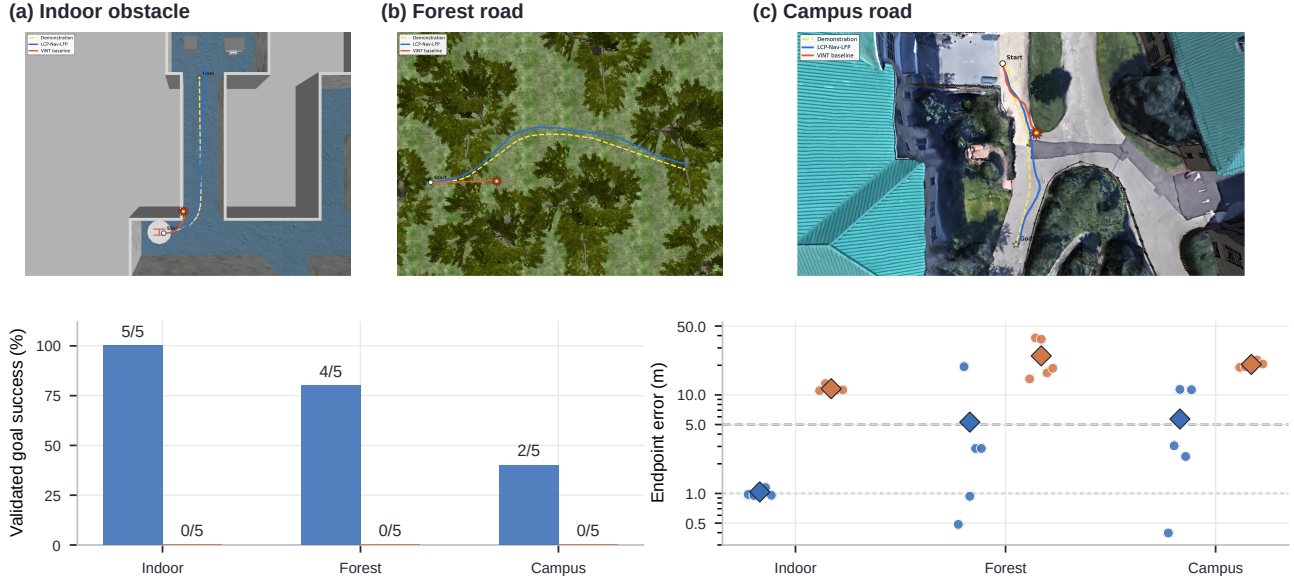


Fig. 3. Scenario-driven closed-loop Gazebo evaluation. The routes stress constrained passage, appearance variation, and terminal drift. Top: (a) indoor-obstacle, (b) forest-road, and (c) campus-road overhead views; the dashed yellow line is the demonstration route, blue and orange are retained LCP-Nav and ViNT trajectories, circles mark starts, stars mark demonstration endpoints, and burst symbols mark failure stops. Bottom: validated arrivals and endpoint error for all five runs per method and scene (diamonds denote means; endpoint error uses a log scale; dashed lines mark 1 and 5 m).

B. Cross-Domain Transfer and Mechanism Analysis

The predictive pathway provides its clearest benefit after domain shift. Relative to the matched ablation, LCP-Nav reduces mean action loss by 1.9% on Go Stanford, 5.6% on ReCon, and 8.3% on SCAND (Fig. 2). ViNT remains stronger on the Go Stanford test set (0.3771 versus 0.4510), whereas LCP-Nav is lower on ReCon (0.3226 versus 0.3399) and SCAND (0.2363 versus 0.3319). Every SCAND seed improves over ViNT; the paired mean difference is -0.0955 , with a 95% exhaustive-bootstrap interval of $[-0.1228, -0.0698]$. The widening full-model gain over the matched ablation indicates that latent consequence prediction contributes most when appearance and motion statistics depart from the training domain.

Go Stanford, ReCon, and SCAND use waypoint spacings of 0.12, 0.25, and 0.38 m. Component metrics in physical units reveal how the aggregate gain changes by domain. On ReCon, LCP-Nav reduces translation RMSE from 0.333 to 0.325 m, while ViNT has lower angle error (0.264 versus 0.290 rad). On SCAND, LCP-Nav has slightly higher translation RMSE (0.555 versus 0.546 m) but lowers angle error from 0.162 to 0.122 rad. The action-loss improvement is therefore not a uniform reduction of every component; it reflects a different balance between translation and heading under shift. Split audits find no trajectory intersections between training and test sets and no overlap between Go Stanford validation and test trajectories.

Mechanism tests explain this pattern. At $\lambda_{\text{sig}} = 0.12$, three candidates produce lower action loss (0.3919) than one (0.4318) or five (0.4685). Removing SIGReg lowers latent-prediction

loss to 0.0023 but collapses the effective rank to 1.02, with 99.7% of variance in the first eigenvalue. Low prediction error is therefore uninformative when distinct observations map to nearly the same latent state. Conversely, cyclically permuting only the imagined endpoints changes approximately 43% of candidate selections across all three datasets, showing that the scorer actively uses the endpoint representation.

The transition also retains useful structure under recursive rollout. One-, two-, and three-step latent MSEs are 1.653, 2.318, and 2.732, compared with 3.743, 4.195, and 4.604 for the matched ablation. At the third step, cosine consistency remains 0.9024 ± 0.0387 for the predictive checkpoints but falls to 0.5098 ± 0.0529 for the ablation. The joint evidence from rank, rollout, and endpoint intervention identifies a functional chain: SIGReg preserves state distinctions, the dynamics propagate them under candidate actions, and the scorer uses the resulting endpoints.

The candidate sweep clarifies the selected K . One candidate leaves the endpoint scorer no alternative, whereas five distribute supervision across more hypotheses and raise action loss. Three candidates provide endpoint variation while keeping each proposal well supervised.

C. Closed-Loop Arrival and Efficiency

LCP-Nav converts the transfer gain into more reliable closed-loop termination. It records 5/5, 4/5, and 2/5 validated arrivals on the indoor, forest, and campus routes, respectively, compared with 0/5 for ViNT in every scene (Fig. 3 and Table I). Its mean endpoint errors are 1.04, 5.30, and 5.70 m, versus 11.56, 24.91, and 20.40 m for ViNT. The overhead views locate the corresponding failures: ViNT stops around the constrained

TABLE I
REPEATED CLOSED-LOOP SIMULATION (MEAN $\pm$ SAMPLE STANDARD
DEVIATION; $n = 5$ RUNS PER METHOD AND SCENE).

Scene	Method	Arrival	Error (m)	Progress (%)
Indoor	LCP-Nav	5/5	1.04 ± 0.10	100.0 ± 0.0
	ViNT	0/5	11.56 ± 0.83	48.2 ± 27.3
Forest	LCP-Nav	4/5	5.30 ± 7.93	79.2 ± 42.1
	ViNT	0/5	24.91 ± 11.43	44.8 ± 11.4
Campus	LCP-Nav	2/5	5.70 ± 5.24	83.8 ± 17.5
	ViNT	0/5	20.40 ± 1.42	100.0 ± 0.0

indoor passage, deviates earlier on the forest route, and reaches the final campus node without terminating near the endpoint.

The scene-wise results separate route traversal from endpoint accuracy. Indoors, LCP-Nav succeeds consistently in the constrained passage. Appearance variation produces one large-error forest run, while the longer campus route retains two high-error terminations. ViNT illustrates the metric mismatch most clearly on campus: it reaches 100% mean topological progress yet finishes 20.40 ± 1.42 m from the endpoint. Under a stricter 1-m threshold, 6/15 LCP-Nav runs and 0/15 ViNT runs remain valid arrivals, preserving the method ordering.

The overhead trajectories distinguish two failure patterns. Indoor and forest ViNT runs deviate or stop early, whereas the campus baseline follows the node sequence but accumulates physical displacement from the endpoint. LCP-Nav passes the indoor bottleneck in every run and terminates most forest and campus runs inside the 5-m criterion; its remaining high-error runs occur on the two longer routes.

Policy-only profiling uses 100 warm-up iterations followed by 500 timed repetitions. LCP-Nav contains 26.77 M parameters, 9.8% fewer than ViNT, and requires approximately 1.164 GFLOPs. On an RTX 4090 with float32 inputs, its p95 latency is 24.3–24.8 ms, compared with 22.6 ms for ViNT; peak allocated memory is 129.6–131.3 MiB, compared with 142.7 MiB. The three latent rollouts therefore add 1.7–2.2 ms while retaining inference above 40 Hz.

V. DISCUSSION

The experiments show that trajectory generation and trajectory selection solve different parts of visual navigation. Direct regression produces a locally plausible action, but it offers no basis for choosing among actions that lead to different terminal states. LCP-Nav makes that choice explicit. The matched ablation attributes the growing cross-domain gain to the predictive pathway, while the endpoint intervention confirms that the selector changes its decision when the imagined consequence changes. Candidate diversity alone is not sufficient:

without a non-collapsed latent state, low prediction error carries little information for ranking.

ViNT fits Go Stanford actions more accurately, whereas LCP-Nav reverses the ordering on ReCon and SCAND despite using the same source data. Together with the matched ablation, this pattern shows that the predictive pathway contributes beyond model capacity. The component metrics show a domain-specific trade-off rather than uniform smoothing: translation and heading change differently across the two target domains.

Closed-loop behavior makes the distinction visible. Indoor results show that endpoint scoring can select stable motion through a constrained passage, and forest results preserve the advantage under appearance variation. The campus route is more demanding: LCP-Nav sharply reduces endpoint error but still records several inaccurate terminations as errors accumulate over the longer route. ViNT reaches every campus node yet stops far from the destination, demonstrating why topological progress must be paired with endpoint error and validated arrival. LCP-Nav achieves this improvement with a small latency increase, so consequence-based selection remains compatible with real-time control.

VI. CONCLUSION

LCP-Nav addresses endpoint drift in visual navigation by comparing the predicted latent consequences of a finite set of local trajectories before execution. On ReCon and SCAND, it achieves lower action loss than ViNT; across three closed-loop Gazebo routes, it reduces endpoint error and increases validated arrivals while sustaining inference above 40 Hz. These results show that endpoint-based consequence comparison can improve action selection without sacrificing real-time control.

REFERENCES

- [1] D. Shah, B. Eysenbach, G. Kahn, N. Rhinehart, and S. Levine, “ViNG: Learning open-world navigation with visual goals,” in *Proc. IEEE Int. Conf. Robot. Autom.*, 2021.
- [2] D. Shah, A. Sridhar, A. Bhorkar, N. Hirose, and S. Levine, “GNM: A general navigation model to drive any robot,” in *Proc. IEEE Int. Conf. Robot. Autom.*, 2023.
- [3] D. Shah *et al.*, “ViNT: A foundation model for visual navigation,” in *Proc. Conf. Robot Learn.*, vol. 229, pp. 711–733, 2023.
- [4] A. Sridhar, D. Shah, C. Glossop, and S. Levine, “NoMaD: Goal masked diffusion policies for navigation and exploration,” arXiv:2310.07896, 2023.
- [5] M. Guerrier, K. Soma, J. Pavlasek, and G. Beltrame, “Can vision foundation models navigate? Zero-shot real-world evaluation and lessons learned,” arXiv:2603.25937, 2026.
- [6] J. Kim, J. Sim, W. Kim, K. Sycara, and C. Nam, “CARE: Enhancing safety of visual navigation through collision avoidance via repulsive estimation,” in *Proc. 9th Conf. Robot Learn.*, 2025.
- [7] Y. Inglin, J. Frey, C. Chen, and M. Hutter, “Less is more: Scalable visual navigation from limited data,” arXiv:2601.17815, 2026.
- [8] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun, “Navigation world models,” arXiv:2412.03572, 2024.
- [9] Y. Dong *et al.*, “Unified world models: Memory-augmented planning and foresight for visual navigation,” arXiv:2510.08713, 2025.
- [10] Z. Sheng, X. Ye, J. Luo, S. Chen, and L. Ren, “ExploreVLA: Dense world modeling and exploration for end-to-end autonomous driving,” arXiv:2604.02714, 2026.
- [11] L. Maes, Q. Le Lidec, D. Scieur, Y. LeCun, and R. Balestrero, “LeWorld-Model: Stable end-to-end joint-embedding predictive architecture from pixels,” arXiv:2603.19312, 2026.